Hospital Appointment Booking System – Complete Project Documentation

🔚 Exit Component – Detailed Explanation

Purpose of the Exit Component

The **Exit component** is responsible for handling the **logout / exit functionality** of the Hospital Appointment Booking System. It ensures that:

- User session data is cleared
- The application returns to the login (intro) page
- No user-specific data remains stored in the browser after exit

This simulates a **real-world logout flow** commonly used in applications.

How Exit Works in This Project (High-Level Flow)

1. User clicks **Exit** from the navigation bar
2. Application navigates to `/Exit`
3. Exit component triggers `onExit` function
4. Application:
   - Clears localStorage
   - Resets React state
   - Redirects user to `/`

Exit Route Configuration (App.js)

JavaScript

```
<Route path="/Exit" element={<Exit onExit={handleExit} />} />
```

Keyword Explanation

- **Route** → Defines a navigable path
- **path="/Exit"** → URL endpoint for exit
- **element={}** → Component to render
- **onExit={handleExit}** → Passing function as prop

👉 This demonstrates **parent-to-child communication using props**.

---

handleExit Function (App.js)

JavaScript

```
function handleExit() {
  localStorage.removeItem("UserName");
```

```
  setName("");
  navigate("/");
}

```

Line-by-Line Explanation

1️⃣ localStorage.removeItem("UserName")

- Deletes stored username
- Prevents session persistence after logout
- Simulates clearing authentication data

2️⃣ setName("")

- Resets React state
- Immediately updates UI
- Prevents old name from appearing again

3️⃣ navigate("/")

- Redirects user to login page
- No page reload (SPA behavior)

Exit Component (Typical Implementation)

JavaScript

```
function Exit({ onExit }) {
  return (
    <div className="exit-container">
      <h2>Thank you for using our service</h2>
      <button onClick={onExit}>Exit</button>
    </div>
  );
}

```

Props Used in Exit Component

	Prop	Type	Purpose
1	onExit	Function	Triggers logout logic

Concept:

- **Function as prop**
- Used to invoke parent-level logic
- Keeps Exit component **stateless & reusable**

Why Logout Logic Is in App.js (Design Decision)

✅ **Correct design practice**

- App.js owns the global state (`name`)
- Exit component should not manage critical state
- Centralized state modification avoids bugs

This demonstrates **separation of concerns**.

React Concepts Demonstrated by Exit Component

- Lifting state up
- Unidirectional data flow
- Stateless functional components
- Programmatic navigation
- Session cleanup

Real-World Mapping (Interview Gold ⭐)

	Real App	This Project
1	Logout API	`handleExit()`
2	Token/session clear	`localStorage.removeItem`
3	Redirect to login	`navigate("/")`

Common Interview Questions – Exit Component

Q1: Why is logout logic not inside Exit component?

Answer: Because session state is managed in App.js. Keeping logout logic centralized ensures consistency and prevents unexpected side effects.

Q2: Why use localStorage for logout?

Answer: localStorage is used to persist user data across page reloads. Clearing it ensures full logout.

Q3: How would this work with a backend?

Answer:

- Call logout API
- Clear JWT/token cookie

- Remove client-side storage
- Redirect to login

Q4: What happens if user refreshes after logout?

Answer: Since localStorage is cleared, no username is retrieved and the login screen is shown.

Q5: How can this Exit component be improved?

Answer:

- Add confirmation popup
- Add loading feedback
- Integrate backend logout
- Prevent direct navigation without login

One-Line Interview Explanation

> "The Exit component handles logout by triggering a parent-managed function that clears local storage, resets React state, and redirects the user—following proper SPA and state management principles."

✅ **This component demonstrates clean architecture, proper state lifting, and real-world logout design — excellent for interviews.**